Formación SQL 360°

Session 5 : Funciones (UDFs)

Facilitador: Flavio Bevilacqua

https://www.linkedin.com/flabevy

Temario de Clase:

- Definición de Funciones
- Beneficios
- Estructura y Sintaxis
- Parametros
- Begin <> End

FUNCIONES

DEFINICIONES

Las funciones customizadas permiten procesar y manipular datos de forma procedural y eficiente. Dichos datos son enviados a través de uno o más parámetros, al momento de invocar la función y retornando un único resultado.

Podemos crear funciones, a la medida de nuestra necesidad, combinando las mismas con funciones ya existentes del lenguaje SQL, para así obtener los resultados deseados tal como necesitamos.

Para crear funciones personalizadas, necesitamos conocer el lenguaje de programación **SQL**, el cual nos permitirá definir una estructura básica que toda función personalizada debe cumplir para operar como tal.

BENEFICIOS

BENEFICIOS

Mejoran la integridad y seguridad de los datos: crear funciones en el servidor, garantiza que, los resultados de los datos procesados serán iguales para cualquier Stack de Programación. Evitando el error humano programando del lado del cliente cuya lógica pueda diferir entre diferentes lenguajes de programación.

Algunos otros beneficios son:

- mejoran la integridad y seguridad de los datos
- optimizan el rendimiento de la base de datos
- otorgan una mejor lectura del código

BENEFICIOS

Optimizan el rendimiento de la base de datos: para complementar el punto anterior, las funciones customizadas de SQL envían los datos procesados por el servidor al cliente, minimizando el tráfico de información.

Otorgan una mejor lectura del código: cuando creas tus propias funciones puedes normalizar o personalizar su nombre, de acuerdo a la convención de nombres de tu país o de la empresa en donde trabajas, haciendo así más fácil la lectura e interpretación del código SQL que utiliza las mismas.

TIPOS DE FUNCION

TIPOS DE FUNCIÓN

En el mundo de la programación, existen tres tipos de funciones:

- Funciones simples
- Funciones con parámetro(s)
- *Funciones con parámetro(s) y retorno*

Particularmente, en SQL, se usa siempre el último tipo de función.

SINTAXIS

SINTAXIS DE FUNCIONES

- **CREATE FUNCTION `nombre_de_la_funcion`** : Esta es la sentencia DDL que nos permite crear una función desde una ventana de *Scripting*. El nombre de la misma se especifica entre comillas simples del tipo **back tick** o, **acento grave**.

- **RETURNS tipoDeDato**: Las funciones devuelven o retornan usualmente algún tipo de dato. Éste puede ser **boolean**, **char** o **number**, entre otros, según lo que estamos procesando.

- Junto a la sentencia **CREATE FUNCTION** debemos especificar qué tipo de datos retornará la misma.

```
CREATE FUNCTION `nombre_de_la_funcion` RETURNS CHAR(60)
...
```

PARAMETROS

PARAMETROS

(param1 INT, param2 INT): las funciones suelen recibir uno o más parámetros de entrada, los cuales provienen de las columnas de una tabla. Debemos especificar el nombre de cada parámetro, junto al tipo de dato de éste y, si son más de uno, debemos separarlos por una coma.

```
CREATE FUNCTION `nombre_de_la_funcion` (param1 INT,
param2 INT) RETURNS CHAR(60)
...
```

TIPOS DE FUNCIONES

TIPOS DE FUNCIONES

Cada función que creamos suele tener diferentes fines.

- Se considera a una rutina como **DETERMINISTIC** (*determinista*) si esta produce el mismo tipo de resultado que el de sus parámetros de entrada. Se utiliza comúnmente con cadena del texto (*string*) o procesamiento matemáticos, aunque no se limita solo a estos resultados.

- Se considera a una rutina como **NO SQL**, cuando esta no utiliza ningún tipo de llamada o invocación de datos a través del lenguaje SQL. O sea, no se llama una sentencia **SELECT**, **UPDATE**, ni cualquier otra sentencia del tipo DML.

- Se considera a una rutina como **READS SQL DATA**, cuando esta función sólo leerá datos de una base de datos. No modificará datos a través de las cláusulas **INSERT**, **UPDATE** o **DELETE**, solo leerá información a través de la sentencia **SELECT**.

BEGIN <> END

CUERPO DE LA FUNCION

BEGIN...END: El cuerpo de la función se define a través de una estructura de inicio y fin. Y es allí, dentro de este cuerpo donde definiremos, a posteriori, el código que le dará vida a la misma.

```
CREATE FUNCTION `nombre_de_la_funcion` (param1 INT, param2 INT)
RETURNS CHAR(60)
BEGIN
...
END
```

Live Coding!

Gracias